Evaluation Validity in Software-Defined Network Intrusion Detection: An Empirical Audit of Controller Observability and Host Generalization on InSDN

Adhyan Jain*
Vellore Institute of Technology
Vellore, India
adhyan.jain2024@vitstudent.ac.in

Atharva Sheersh Pandey*
Vellore Institute of Technology
Vellore, India
atharva.sheersh2024@vitstudent.ac.in

Kshitij Manikshete*
Vellore Institute of Technology
Vellore, India
kshitij.rahul2024@vitstudent.ac.in

***Abstract*—Machine learning for Software-Defined Network (SDN) intrusion detection is predominantly evaluated on benchmark flow CSVs derived offline from packet captures at end hosts using tools such as CICFlowMeter. Whether an SDN controller physically observes telemetry resembling these 5-tuple flow records during operational forwarding is rarely scrutinized. In this empirical audit, we investigate evaluation validity on InSDN, the primary public SDN benchmark providing synchronized control-plane captures, along two complementary axes: physical controller observability and topological data leakage.**

First, analyzing all 44 packet captures of InSDN's ONOS OpenFlow 1.3 control channel (comprising 11 benign and 33 attack sessions across seven attack classes, totaling 7 381 679 OpenFlow messages), we find that in every flow-statistics reply and in all 3 827 FlowMods, flow table entries match exclusively on `eth_type` or `(in_port, eth_src, eth_dst)` under its deployed reactive forwarding configuration. None matches IP addresses or transport ports. Consequently, the benchmark's labelled 5-tuple flow records represent counters the controller cannot read under this setup. Instead, the controller receives a dense PacketIn stream: across 9 of 12 attack scenarios, the controller receives 0.68 to 1.29 PacketIns per data packet recorded at the attacked target host link. As an empirical feasibility probe, small multi-layer perceptrons operating strictly on controller-visible window features achieve a balanced accuracy of 0.946 on unseen captures under capture-grouped cross-validation (compared to 0.616 for standard PacketIn-rate heuristics) and 0.89–1.00 across withheld attack families. However, we demonstrate that these detection scores cannot be interpreted as deployable attack detection: because InSDN isolates benign and attack traffic into separate recording sessions and CSV labels cover only partial traffic, a trivial "any IP PacketIn" heuristic achieves 0.995 balanced accuracy within attack sessions under strict ground-truth matching. A complementary Mininet/Ryu testbed case study demonstrates that forwarding application choices dictate observability: per-flow forwarding generates about 10 400 TCP PacketIns per run compared to about 12 for MAC-pair forwarding (a 665× to 1 752× increase), while a 2 s idle timeout reduces flow table visibility from 153.1 ± 2.4 to 36.9 ± 0.5 entries per reply. Ryu's stock `simple_switch_13` similarly installs only MAC-pair entries (86 of 91 FlowMods; 0 matching IP or transport fields).

Second, we audit evaluation protocol validity. Across InSDN (343 889 flows) and an external reference benchmark, CICIDS2017 (2 100 814 flows), standard random flow splits yield near-perfect balanced accuracies of 0.9998 and 0.9917, respectively. In contrast, holding out source hosts drops performance to 0.907 ± 0.085 on InSDN (a nine percentage-point drop) and collapses to 0.641 ± 0.056 on CICIDS2017 (a 35 percentage-point drop). Models trained solely on IP addresses, ports, and Flow IDs achieve at least 0.9999 on both datasets, confirming that network identifiers alone provide trivial class separability without requiring statistical traffic features. Although 48.7% of InSDN's random-split test rows duplicate training rows (20.3% in CICIDS2017), complete deduplication leaves random-split accuracy unchanged at 0.9998 (184,607 rows remaining), while class-covered host-held-out splits score 0.939 (0.812 after deduplication) and triple-grouped splits score 0.961 (0.984 after deduplication). We show that the held-out-host variance is consistent with being driven by acute host concentration: 122,307 of InSDN's 122,868 source IPs appear in only one flow, with just eight hosts accounting for 215,404 rows (three being attackers), resulting in widely dispersed split scores swinging from 1.00 down to 0.66. Holding out the primary attacker drops recall to 51.6% (DoS recall 34.7%) while flagging 37.9% of flows from the largest benign host. A targeted audit of 12 representative SDN-IDS papers reveals that ten were read in full text; none reports a host-grouped split, and three utilize identifier columns as model inputs. Furthermore, four generic feature recipes approximating proposals in the literature on InSDN achieve at least 0.9998 random-split balanced accuracy but drop to 0.856–0.943 under host-grouped evaluation. We conclude with concrete methodological guidelines and an evaluation checklist for future SDN-NIDS research.

***Index Terms*—Software-defined networking, intrusion detection systems, OpenFlow, ONOS, InSDN, control-plane telemetry, evaluation methodology, data leakage, generalization.**

*All three authors contributed equally to this work.

I. Introduction

Software-Defined Networking (SDN) decouples network control logic from underlying forwarding hardware, enabling centralized programmability and dynamic flow management [1], [2]. Motivated by the controller's global architectural view, a substantial literature proposes machine learning (ML) and deep learning Network Intrusion Detection Systems (NIDS) designed to secure SDN environments [3], [4], [5].

Despite widespread interest, a critical methodological disconnect separates theoretical proposals from operational networking realities. In practice, SDN intrusion detection research rarely interacts with an active SDN controller. Instead, researchers predominantly evaluate machine learning models

on offline comma-separated value (CSV) benchmarks, such as InSDN [3] and CICIDS2017 [6]. These benchmark tables consist of bidirectional 5-tuple flow summaries computed by feature extractors like CICFlowMeter [7] operating directly on full-packet captures obtained from network taps at end hosts. Standard pipelines train high-capacity classifiers on 80+ statistical features—including inter-arrival time moments, bidirectional packet length distributions, and cumulative TCP flag counts—and consistently report detection accuracies exceeding 99.5% [8], [9], [10].

This prevalent methodology suffers from two foundational threats to validity, illustrated conceptually in Fig. 1:

1) **The Controller Observability Disconnect:** An OpenFlow controller does not have native access to packet-level timing moments or fine-grained per-flow payload metrics. Under standard OpenFlow 1.3 reactive forwarding [11], the controller receives `PacketIn` messages for initial flow misses, issues `FlowMod` entries, and periodically polls aggregate flow statistics counters. Whether the controller's installed flow table entries actually capture fine-grained 5-tuple counters depends on the controller's forwarding application. If the controller installs wildcard or MAC-pair rules, granular 5-tuple statistical features cannot be constructed at the controller without either altering flow-table granularity (risking TCAM exhaustion) or streaming full payloads (risking control-channel saturation).
2) **The Evaluation-Validity and Topology Leakage Disconnect:** In security machine learning, randomized train/test splits are known to suffer from severe data leakage when applied to benchmark datasets with repetitive traffic or homogenous host assignments [12], [13], [14], [15]. When flows from the same IP address or session appear in both training and test partitions, models learn spurious host-specific shortcut representations rather than generalizable attack signatures [16]. In SDN topologies with highly concentrated host structures, reported near-perfect accuracies may simply reflect the model's ability to recognize previously seen hosts rather than its capability to detect novel attackers.

While prior studies have highlighted feature calculation discrepancies [17], [18], [19], [20] or identified data smells in general non-SDN datasets [15], the specific manifestation of these issues within SDN control-plane environments remains largely unquantified. Crucially, the InSDN dataset release included not only offline flow CSVs but also the raw PCAP files of the ONOS controller's OpenFlow 1.3 control channel across all evaluated scenarios [3]. This provides a unique, empirical opportunity to examine what an SDN controller actually observes during benchmark execution and to systematically audit evaluation validity.

In this work, we conduct an extensive empirical measurement study into controller observability and evaluation validity in SDN intrusion detection. We make seven key contributions:

1) **A measured controller observability gap (§IV):** We reassemble and decode all 44 OpenFlow 1.3 control-channel packet captures shipped with InSDN. Across all flow-statistics replies, only two match structures appear: default `eth_type` rules and reactive `(in_port, eth_src, eth_dst)` entries. Across all 3 827 FlowMods sent by the ONOS controller, none matches IP addresses or transport ports. Thus, the benchmark's labelled 5-tuple flows are physically unobservable as controller counters under this configuration. Instead, the controller receives a dense PacketIn stream: across 9 of 12 attack scenarios, the controller receives 0.68 to 1.29 PacketIns per data packet seen at the target host link.
2) **A reproducible control-channel telemetry pipeline (§V):** We construct an OpenFlow 1.3 parser with resilient desynchronization recovery, parsing 7 381 679 control-plane messages across the 44 captures. We engineer 22 streaming window features ($W{=}1$ s and 5 s) derived solely from controller-observable primitives, with ground-truth labels strictly mapped from InSDN's flow records.
3) **Controller-observable intrusion detection feasibility probe (§VI):** We evaluate lightweight classifiers operating solely on controller-visible window features, reaching a balanced accuracy of 0.946 on unseen captures under capture-grouped cross-validation, outperforming standard PacketIn-rate threshold heuristics (0.616) and evaluating generalization across withheld attack families (0.89–1.00).
4) **A validity analysis of controller-view scores (§VII):** We critically examine whether these controller-side scores reflect genuine attack detection. We discover that because InSDN segregates benign and attack traffic into separate recording sessions, within attack sessions a trivial "any IP PacketIn" heuristic scores 0.995 balanced accuracy. Furthermore, residual session probes demonstrate that session-level artifacts cannot be untangled from attack traffic in non-interleaved benchmark designs.
5) **A quantified evaluation-validity audit across two datasets (§VIII):** Using a standardized XGBoost pipeline without identifier attributes, we show that random flow splits achieve 0.9998 balanced accuracy on InSDN (343 889 flows) and 0.9917 on CICIDS2017 (2 100 814 flows). When evaluating on held-out source hosts, accuracy drops to 0.907 ± 0.085 on InSDN and collapses to 0.641 ± 0.056 on CICIDS2017. Models trained strictly on IP addresses, ports, and Flow IDs achieve at least 0.9999, demonstrating that network identifiers alone provide trivial class separability in these benchmarks.
6) **Root-cause analysis of topology concentration and deduplication (§VIII):** Our empirical analysis shows that InSDN's held-out-host variance is driven by acute host concentration: 122,307 of InSDN's 122,868 source IPs appear in only one flow, with just eight hosts accounting for 215,404 rows. In a Leave-One-Host-Out (LOHO) evaluation, holding out the primary attacker drops attack recall to 51.6% (DoS recall 34.7%) while flagging 37.9% of flows from the largest benign host. Deduplicating InSDN (removing 48.7% redundant rows) leaves random-split accuracy at 0.9998 (184,607 rows remaining), while class-covered host splits yield 0.939 (0.812 after deduplication) and triple-grouped splits score

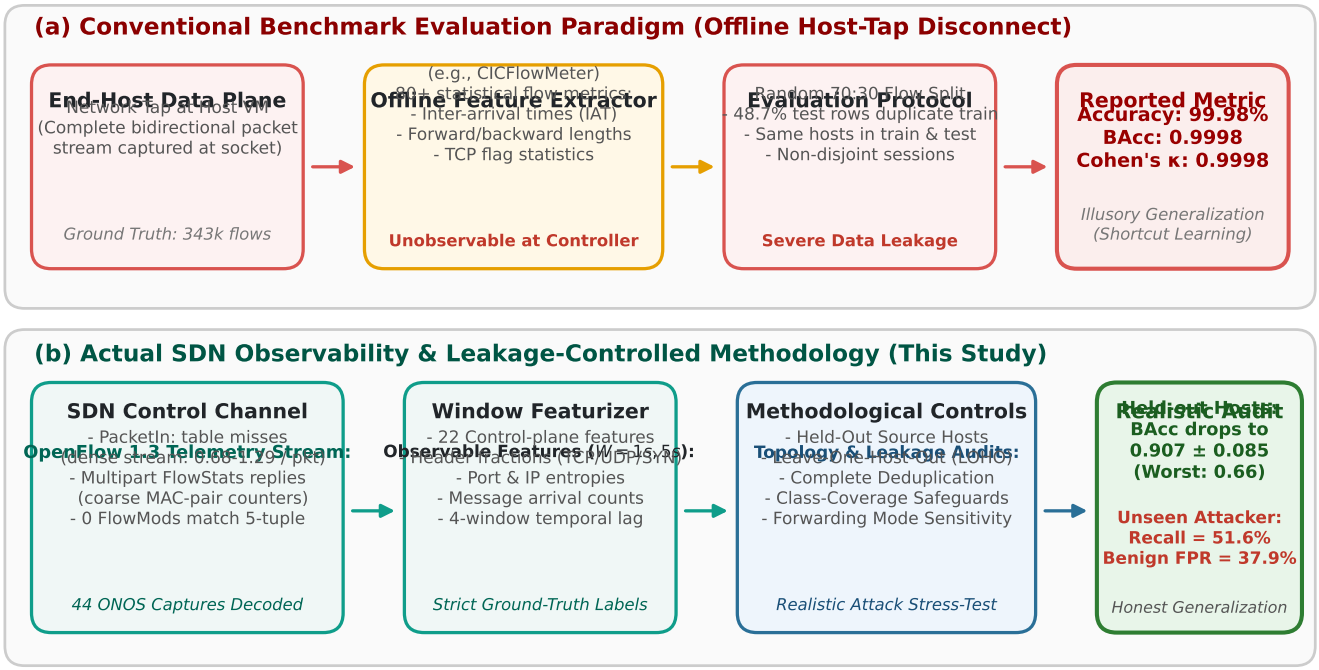


Fig. 1. The foundational disconnect in SDN intrusion detection research. (a) Conventional benchmark evaluations extract 80+ offline statistical features from host-side socket captures and apply randomized train/test splits, resulting in heavy cross-split host overlap, 48.7% duplicate rows, and illusory 99.98% accuracy. (b) In operational SDN deployments, the controller observes only OpenFlow 1.3 control-plane telemetry (PacketIn messages and coarse MAC-pair FlowStats), where zero FlowMods match 5-tuple fields. When evaluated under rigorous host-disjoint and deduplicated protocols, detection performance on unseen attackers drops sharply.

0.961 (0.984 after deduplication).

7) **Targeted literature audit, recipe replications, and forwarding mechanics (§IX, §X):** We conduct a targeted audit of 12 representative SDN-IDS papers (ten read in full text; none reports a host-grouped split; three use identifiers as model inputs). Evaluating four generic feature recipes approximating proposals in the literature on InSDN shows that all achieve at least 0.9998 random-split balanced accuracy but drop to 0.856–0.943 under host-grouped splits. A Mininet/Ryu testbed case study confirms that forwarding granularity alters PacketIn rates by orders of magnitude (10 422 vs. 12.3 TCP PacketIns per run), and stock Ryu installs only MAC-pair entries (86 of 91 FlowMods).

The remainder of this paper is organized as follows. Section II reviews background and related work. Section III formalizes the threat model, telemetry semantics, and problem formulation. Section IV examines controller observability in InSDN. Section V details the controller-observable detection pipeline, and Section VI presents detection results. Section VII analyzes benchmark session confounds. Section VIII presents the evaluation-validity audit on InSDN and CICIDS2017. Section IX details the Mininet/Ryu forwarding-application case study. Section X presents the targeted literature audit and recipe replications. Section XI outlines methodological guidelines and an evaluation checklist. Section XII discusses limitations, and Section XIII concludes the paper.

II. Background and Related Work

A. SDN Control-Plane Mechanics and Telemetry

In an OpenFlow 1.3 network [11], switches maintain flow tables containing match rules, priorities, timeouts, and actions. When a packet arrives at an OpenFlow switch without matching an existing flow entry, it triggers a table-miss event. Under reactive forwarding configurations, the switch encapsulates the packet header (or the entire frame) into an asynchronous `OFPT_PACKET_IN` message and transmits it across the secure control channel to the controller. The controller's forwarding logic processes the packet headers, determines the forwarding path, and responds with an `OFPT_PACKET_OUT` message to forward the buffered frame. Concurrently, the controller may issue an `OFPT_FLOW_MOD` message to install a matching flow entry in the switch's flow table.

Subsequent packets belonging to the same flow match the installed entry directly in hardware or kernel space (such as Open vSwitch datapath), bypassing the controller entirely. The controller can monitor data-plane activity by issuing multipart `OFPMP_FLOW` requests to poll flow statistics. Switches respond with `OFPMP_FLOW` replies containing packet and byte counters for currently active flow entries. When an entry exceeds its idle or hard timeout, the switch evicts it and may send an `OFPT_FLOW_REMOVED` notification.

Crucially, the granularity of information visible to the controller via flow statistics is entirely dictated by the match fields specified in preceding FlowMods by the forwarding application. If an application installs coarse MAC-pair entries (`in_port`,

`eth_src, eth_dst`), all transport connections and IP flows between those two endpoints aggregate into a single counter. Conversely, if an application installs granular 5-tuple entries, flow table occupancy increases dramatically, exposing switches to flow-table saturation attacks.

B. Pitfalls in Machine Learning Security and Network Datasets

The vulnerability of supervised machine learning to subtle experimental artifacts in computer security was seminalized by Sommer and Paxson [12]. More recently, Arp et al. [13] systematized ten common pitfalls in security ML, emphasizing sampling bias, data snooping, and spurious correlations.

In network intrusion detection specifically, benchmark datasets have faced substantial scrutiny. Engelen et al. [14] discovered critical flow construction, labeling, and TCP sequence errors in CICIDS2017, explicitly warning that non-statistical identifier columns (such as Flow IDs, IP addresses, and timestamps) allow classifiers to memorize host identities rather than attack patterns. Lanvin et al. [21] and Liu et al. [22] demonstrated that correcting label and orchestration bugs significantly shifts published model rankings. Flood et al. [15] surveyed seven benchmark datasets (excluding SDN datasets) and introduced the concept of "bad design smells," proving that standard randomized train/test splits result in widespread cross-split leakage due to synthetic traffic homogeneity. Roy et al. [16] demonstrated analogous shortcut learning dynamics on IoT intrusion benchmarks.

C. Intrusion Detection in SDN Environments

Early SDN security research focused on leveraging control-plane events to mitigate volumetric denial-of-service attacks. Braga et al. [23] proposed lightweight DDoS detection using 6-tuple flow statistics polled from NOX controllers. Mousavi and St-Hilaire [24] utilized PacketIn entropy variations for early DDoS detection. To protect the SDN control plane against saturation attacks, architectures such as AVANT-GUARD [25], FloodGuard [26], and SPHINX [27] introduced connection migration and flow-table constraint validation.

With the release of benchmark datasets such as InSDN [3], research shifted heavily toward applying advanced deep learning models to flow-level CSV features. Ataa et al. [4] evaluated CNN-LSTM and Transformer architectures on InSDN, reporting accuracies up to 99.02%. Ahmad et al. [8] proposed OptiXGB-IDS, combining WGAN-GP oversampling with XGBoost to achieve 99.87% accuracy on InSDN. Sajid et al. [10] reported 99.98% accuracy using optimizer-equipped CNN-MLPs. Ashikuzzaman et al. [5] introduced SDNGuard-Stack, reporting 99.98% accuracy using stacked ensemble learning. However, as we document in Section X, these works evaluate models exclusively on offline flow records and rely on randomized flow-level splits, leaving their operational validity unexamined.

III. Threat Model and Problem Formulation

A. System Architecture and Telemetry Semantics

Consider an SDN architecture comprising a set of OpenFlow switches $\mathcal{S}$ controlled by a logically centralized controller $\mathcal{C}$ over OpenFlow 1.3 control channels. The data plane carries traffic between end hosts $\mathcal{H} = \mathcal{H}_{\text{benign}} \cup \mathcal{H}_{\text{attack}}$.

The control channel between switch $s \in \mathcal{S}$ and controller $\mathcal{C}$ carries a discrete sequence of timestamped OpenFlow messages:

$$\mathcal{M} = \{(t_i, \tau_i, m_i)\}_{i=1}^{N}, \tag{1}$$

where $t_i \in \mathbb{R}^+$ denotes the arrival timestamp, τ_i denotes the message type belonging to the control-plane set {PacketIn, PacketOut, FlowMod, FlowRemoved, StatsReply}, and m_i represents the message payload. When τ_i = PacketIn, m_i encapsulates the arriving frame header $h_i = (\text{mac}_{\text{src}}, \text{mac}_{\text{dst}}, \text{eth_type}, \text{ip}_{\text{src}}, \text{ip}_{\text{dst}}, p_{\text{src}}, p_{\text{dst}}, \text{proto})$. When τ_i = StatsReply, m_i conveys flow table records $e_{i,j} = (\text{match}_{i,j}, \text{pkts}_{i,j}, \text{bytes}_{i,j}, \text{dur}_{i,j})$.

B. Problem Formulation: Streaming Window Detection

Rather than assuming omniscient offline flow records, a controller-side detector operates on discrete time windows of duration W. For each window k spanning $[kW, (k+1)W)$, the detector computes a feature vector:

$$\mathbf{x}_k = \phi(\mathcal{M} \cap [kW, (k+1)W)) \in \mathbb{R}^d, \tag{2}$$

where $\phi(\cdot)$ extracts observable control-plane metrics. Given a history of H preceding windows, the model infers a binary or multi-class label $\hat{y}_k \in \{0, 1\}$.

C. Data Partitioning Formulations

To evaluate model generalization, we formalize four partitioning strategies over a dataset $\mathcal{D} = \{(\mathbf{x}_n, y_n, h_n)\}_{n=1}^{|\mathcal{D}|}$, where $h_n \in \mathcal{H}$ denotes the host associated with record n:

1) **Random Flow Split:** Partitions $\mathcal{D}$ into disjoint subsets $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{test}}$ via uniform random sampling. Crucially, the host sets overlap heavily: $\text{Hosts}(\mathcal{D}_{\text{train}}) \cap \text{Hosts}(\mathcal{D}_{\text{test}}) \approx \mathcal{H}$.
2) **Held-Out Host Split:** Partitions the host space $\mathcal{H}$ into disjoint subsets $\mathcal{H}_{\text{train}}$ and $\mathcal{H}_{\text{test}}$ such that $\mathcal{H}_{\text{train}} \cap \mathcal{H}_{\text{test}} = \emptyset$. All records belonging to $\mathcal{H}_{\text{train}}$ form $\mathcal{D}_{\text{train}}$, and records belonging to $\mathcal{H}_{\text{test}}$ form $\mathcal{D}_{\text{test}}$.
3) **Leave-One-Host-Out (LOHO):** For each host $h^* \in \mathcal{H}$, the test set is defined strictly as $\mathcal{D}_{\text{test}} = \{(\mathbf{x}_n, y_n, h_n) \mid h_n = h^*\}$, while the training set comprises all remaining hosts $\mathcal{D}_{\text{train}} = \{(\mathbf{x}_n, y_n, h_n) \mid h_n \neq h^*\}$.
4) **Time-Ordered Split:** For a temporal dataset, records are ordered by timestamp $t_1 \leq t_2 \leq \cdots \leq t_N$, and a cutoff T_{split} defines $\mathcal{D}_{\text{train}} = \{(\mathbf{x}_n, y_n) \mid t_n < T_{\text{split}}\}$ and $\mathcal{D}_{\text{test}} = \{(\mathbf{x}_n, y_n) \mid t_n \geq T_{\text{split}}\}$.

D. Threat Model and Scope

Defender Capabilities: The defender deploys an intrusion detection module co-located with or tapping the SDN controller interface. The detector inspects solely OpenFlow control-plane traffic $\mathcal{M}$. It possesses no packet taps on data-plane switch ports and cannot read host-side sockets. The forwarding application running on the controller is assumed fixed and known.

Attacker Model: The attacker controls one or more compromised end hosts in the network. The attacker initiates

network attacks including volumetric flooding (DoS/DDoS), port scanning, service probing, brute-force authentication, web application exploitation, and botnet command-and-control communication. The attacker does not compromise the controller kernel or directly forge control-channel TLS certificates.

Out of Scope: Real-time inline mitigation latency, hardware switch TCAM exhaustion, and adversarial evasion of control-plane telemetry features are outside the scope of this study.

IV. What the Controller Sees in InSDN

A. Dataset Inventory and Capture Reassembly

InSDN [3] was generated in a virtualized testbed using Mininet and Open vSwitch (OVS) connected to an ONOS controller. Traffic targets a Metasploitable-2 virtual machine or the OVS switch itself. In addition to releasing CSV flow files, the authors published raw PCAP archives recorded at the target hosts and at the ONOS controller's OpenFlow interface.

We fetched all 44 ONOS controller packet captures directly from the institutional repository, verifying each archive against published SHA-256 digests. As summarized in Table I, the 44 captures encompass 11 benign sessions and 33 attack sessions spanning seven distinct attack classes.

B. Match Field Granularity and FlowMod Analysis

To determine what flow entries were active at the switches, we parsed and decoded every OpenFlow 1.3 multipart flow-statistics reply across all 44 controller captures. Over the entire dataset, only two distinct match structures appear:

1) `eth_type`: Default punting rules installed by ONOS (80 464 entry observations).
2) `(in_port, eth_src, eth_dst)`: Reactive MAC-pair forwarding rules (24 437 entry observations).

Not a single entry in any flow-statistics reply matches on source IP, destination IP, transport protocol, or transport port numbers. A 5-tuple flow—the fundamental unit labelled in InSDN's CSV—is never represented as an individual counter within the controller's flow table.

To verify whether transient 5-tuple entries could have been installed and deleted between polling intervals, we analyzed all 3 827 FlowMod messages transmitted by the ONOS controller. The results are decisive: 3 811 FlowMods match strictly on `(in_port, eth_dst, eth_src)`, 16 match on `eth_type`, and exactly zero FlowMods specify IP addresses or transport ports. The ONOS forwarding application operated purely as an Ethernet learning switch.

C. Dynamics of the Controller PacketIn Stream

Because the flow table entries match only MAC pairs, the controller relies on PacketIn messages for unmatched frames. In Table II and Fig. 2, we pair target-side packet counts with controller-side PacketIn volume across all attack scenarios.

In 9 of 12 attack scenarios, the controller receives between 0.68 and 1.29 PacketIns per data packet recorded at the target. Two Metasploitable-2 scenarios show 3.9 and 15.7. The OVS DDoS scenario shows 0.08 (6.5M target packets against 541k PacketIns), consistent with control-channel saturation under a spoofed flood; we did not test this directly. The two sides are separate recordings and their alignment is unverified, so ratios are indicative, not exact per-packet counts. The Normal group has no target-side captures, so we have no such ratio for benign traffic. Because each PacketIn carries the packet's headers, the 5-tuple is visible to the controller per packet, but never as a per-flow counter in the switch flow table.

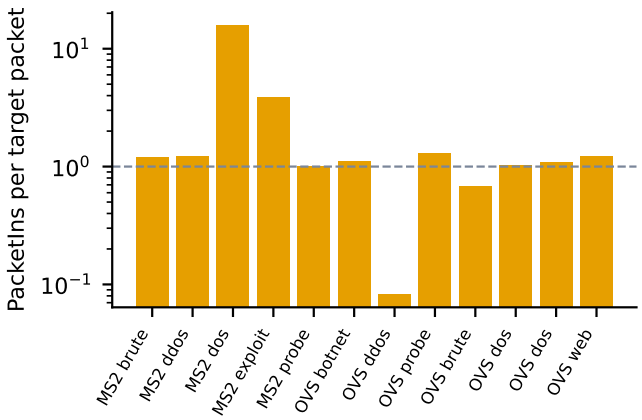


Fig. 2. PacketIns received by the ONOS controller per packet seen at the target host across InSDN attack scenarios (MS2: Metasploitable-2 target, OVS: OVS switch target). The dashed line marks parity (one PacketIn per target packet). Data: `gap_attack_v2.json`.

Implication. A model trained on InSDN's CSV consumes statistics (inter-arrival moments, per-direction length moments, flag counts per 5-tuple) that this controller's flow tables never recorded. In principle, because PacketIns expose packet headers, an SDN monitor could attempt to reconstruct 5-tuple flow summaries in software from the PacketIn stream. However, such reconstruction faces operational constraints under reactive forwarding: in volumetric flood scenarios like OVS DDoS (where 6.5M target packets yielded only 541 094 PacketIns, an 8% sampling ratio), the PacketIn stream captures only a small fraction of data-plane packets, so software-reconstructed flow moments would be incomplete without dedicated monitoring rules or data-plane telemetry (e.g., sFlow or In-band Network Telemetry). We did not evaluate software flow reassembly from PacketIns and leave this to future work; instead, our controller-side detector evaluates the signal the controller actually received: the PacketIn header stream plus coarse MAC-pair counters.

V. Controller-Observable Intrusion Detection

A. Streaming OpenFlow 1.3 Parser

We implemented a streaming OpenFlow 1.3 parser designed to reassemble TCP streams on port 6653. The parser handles message boundary segmentation and employs heuristic desynchronization recovery. If a message header is corrupted or desynchronized, the parser scans forward until it discovers two consecutive valid OpenFlow headers. Across all 44 captures, the parser processed 7 381 679 messages, requiring only 134 resynchronizations and discarding 12 217 bytes in total.

B. Feature Engineering

For consecutive non-overlapping windows of duration $W \in \{1\text{s}, 5\text{s}\}$, we extract 22 controller-observable features:

- **Control Message Counts (5):** Total counts of PacketIn, PacketOut, FlowMod, FlowRemoved, and FlowStats replies.
- **Header Diversity and Distributions (9):** Distinct source MACs, destination MACs, source IPs, destination ports; fractions of TCP, UDP, ICMP, and ARP traffic; and mean frame length.
- **Protocol and Entropy Metrics (3):** TCP SYN-only packet fraction, Shannon entropy of source IPs, and Shannon entropy of destination ports.
- **Flow Table Counter Deltas (5):** Number of active entries reported in statistics replies, sum and maximum of packet deltas, and sum and maximum of byte deltas between consecutive polls.

To capture temporal dynamics, the feature vector concatenates the current window with three preceding windows ($H = 4$), applying a $\log(1 + x)$ transformation to normalize dynamic ranges.

C. Strict Ground-Truth Window Labeling

Assigning ground-truth labels to controller windows requires precision. Marking every window within an attack capture as malicious would mislabel idle periods. Instead, we match the 5-tuple header of every PacketIn message against InSDN's ground-truth CSV flow records.

A window is labelled as an attack window if and only if it contains at least one PacketIn matching a known attack flow. In attack captures, windows containing unmatched IP traffic are marked as ambiguous and excluded from training and evaluation. Benign captures retain all windows. As shown in Table I, this strict protocol yields a sparse set of verified attack windows for low-volume classes (e.g., 6 windows for Brute Force, 12 for Exploit).

D. Evaluated Models and Baselines

We evaluate several detector architectures:

- **Multi-Layer Perceptron (MLP):** A fully connected network (64–32 units, ReLU activation, dropout rate 0.2, class-weighted cross-entropy loss, Adam optimizer, 40 epochs).
- **Linear Classifier:** Logistic regression with L2 regularization operating on the same historical feature inputs.
- **Single-Window MLP:** An MLP evaluating only the current 1 s window without temporal history.
- **Feature-Ablated MLPs:** Models trained exclusively on PacketIn-derived features or flow-statistics features.
- **Threshold Heuristics:** Univariate decision thresholds fitted on training folds (PacketIn rate, normalized burst rate, source-IP entropy, destination-port entropy).
- **Gradient Boosted Trees (XGBoost):** An optimized XGBoost baseline trained on the GPU using the same window features.
- **Host-Side Flow Baseline:** XGBoost trained on InSDN's 78 numeric CICFlowMeter features under standard 70:30 flow splits.

E. Validation Protocols

All evaluations enforce strict capture-level separation: no capture file appears in both training and testing sets. We report metrics across 10 random initialization seeds:

- **Protocol A (Grouped 5-Fold CV):** Captures are partitioned into five stratified folds.
- **Protocol B (Withheld Attack Families):** Evaluates zero-day generalization by holding out entire attack families (DoS/DDoS pooled as flooding).
- **Protocol C (Cross-Deployment Transfer):** Training on Metasploitable-2 sessions and testing on OVS sessions, and vice-versa.

We report Balanced Accuracy (BAcc), Recall, and False Positive Rate (FPR), reporting mean and 95% confidence intervals (t-distribution).

VI. Empirical Detection Results

A. Performance on Unseen Captures

Table III presents detection results on unseen captures under 5-fold cross-validation ($W{=}1$ s). The controller-observable MLP achieves a balanced accuracy of 0.946 with a false positive rate of 0.001. Logistic regression performs competitively at 0.939, and the single-window MLP reaches 0.934, indicating that temporal history provides marginal separation gain in this feature space.

In contrast, standard SDN heuristics perform poorly: the PacketIn-rate threshold achieves only 0.616 balanced accuracy (FPR 0.561), failing because benign interactive sessions also generate bursty PacketIn streams. Entropy thresholds perform substantially better (0.839–0.852). Ablation confirms that PacketIn features drive performance (0.943 alone), while flow-statistics features alone achieve only 0.699 with an FPR of 0.284, consistent with coarse MAC-pair aggregation.

A cluster bootstrap over captures (Table IV) yields a pooled BAcc of 0.984 [0.926, 0.995] for the MLP. A label-shuffle sanity check produces a balanced accuracy of 0.512 ± 0.054 at 1 s and 0.508 ± 0.067 at 5 s, i.e., chance level; this confirms that the featurization pipeline does not inadvertently encode label information directly, though it does not address session-level confounding.

B. Withheld Attack Families and Cross-Deployment Transfer

Under leave-one-attack-family-out testing (Table V), the controller-side MLP demonstrates strong generalization on volumetric and scanning attacks, achieving 0.936 on floods, 0.997 on probes, and 0.904 on web attacks, outperforming individual threshold heuristics. On sparse minority classes, thresholds or linear models match or exceed it (e.g., botnet threshold 0.965 vs. MLP 0.932; logistic regression 0.997 on exploit). In cross-deployment evaluations (Table VI), training on OVS sessions and testing on Metasploitable-2 yields 0.990 balanced accuracy, while the reverse direction achieves 0.905

TABLE I
CONTROLLER CAPTURES (1 S WINDOWS). UNDER STRICT LABELS, AMBIGUOUS WINDOWS OF ATTACK CAPTURES ARE EXCLUDED ("KEPT").

Class	Captures	Windows	Kept	Attack win.	PacketIns
Normal	11	33,283	33,283	0	789,569
DoS	11	861	805	456	845,362
DDoS	6	499	184	55	1,340,347
Probe	4	5,968	2,921	1,965	621,986
Brute force	4	604	359	6	5,564
Exploit	4	88	71	12	332
Web	3	636	275	12	1,199
Botnet	1	160	152	83	1,002

TABLE II
TARGET-SIDE TRAFFIC VS. CONTROLLER OBSERVATIONS PER SCENARIO (RATIO = PACKETINS PER TARGET PACKET).

Group	Scenario	Target pkts	5-tuples	PacketIns	Ratio
Metasp.	Brute force	1,311	295	1,574	1.20
Metasp.	DDoS	652,495	651,083	799,253	1.22
Metasp.	DoS	13,078	989	205,198	15.69
Metasp.	Exploit	86	18	332	3.86
Metasp.	Probe	507,650	51,150	510,194	1.01
OVS	Botnet	902	164	1,002	1.11
OVS	DDoS	6,510,923	6,508,402	541,094	0.08
OVS	Probe	86,705	79,021	111,792	1.29
OVS	Brute force	5,892	1,112	3,990	0.68
OVS	App. DoS	299,625	41,738	308,681	1.03
OVS	Net. DoS	304,886	20,979	331,483	1.09
OVS	Web	987	176	1,199	1.21

TABLE III
UNSEEN CAPTURES, GROUPED 5-FOLD CV, W=1 S, STRICT LABELS, 10 SEEDS (MEAN±95% CI OVER SEEDS).

Method	BAcc	Recall	FPR
MLP, all features	0.946±0.016	0.893	0.001
MLP, PacketIn features only	0.943±0.015	0.887	0.002
Logistic regression (history)	0.939±0.016	0.887	0.009
MLP, single window	0.934±0.015	0.870	0.003
Thr. source-IP entropy	0.852±0.029	0.763	0.059
Thr. dst-port entropy	0.839±0.018	0.736	0.059
MLP, flow-stats features only	0.699±0.036	0.683	0.284
Thr. PacketIn burst	0.643±0.008	0.912	0.627
Thr. PacketIn rate	0.616±0.011	0.793	0.561
MLP, within attack sessions	0.942±0.016	0.893	0.008
macro-averaged over captures	0.906±0.009	–	–
Trivial: "any IP PacketIn" (same windows)	0.995	–	–
macro over captures	1.000	–	–

TABLE IV
POOLED WINDOW-COUNT BACC WITH A CAPTURE-LEVEL CLUSTER BOOTSTRAP (2000 RESAMPLES, STRATIFIED BY CLASS).

Method	Pooled BAcc	95% CI
MLP, all features	0.984	[0.926, 0.995]
Logistic regression	0.979	[0.921, 0.991]
MLP, single window	0.979	[0.905, 0.993]
Thr. source-IP entropy	0.923	[0.783, 0.959]
Thr. dst-port entropy	0.926	[0.792, 0.953]
Thr. PacketIn rate	0.467	[0.245, 0.817]

(with logistic regression reaching 0.928). Extending the window length to W=5 s yields consistent performance (MLP 0.958, logistic regression 0.961, XGBoost 0.952).

C. Comparison with Host-Side Flow Baseline

For contrast, applying XGBoost to InSDN's host-side CICFlowMeter features (343 889 rows, 78 numeric features) under standard randomized 70:30 flow splits yields a balanced accuracy of 0.9998 ± 0.00003. However, this near-perfect accuracy should not be viewed as evidence of superior detection; rather, as demonstrated below, it reflects the inherent ease and data leakage of randomized flow splits on few-host datasets.

VII. VALIDITY ANALYSIS: DISENTANGLING ATTACKS FROM SESSION ARTIFACTS

A. The Trivial IP PacketIn Rule

InSDN's experimental protocol recorded benign and attack traffic in separate Mininet sessions. In attack sessions, background benign activity was not generated concurrently. Consequently, strict labeling creates an extreme experimental artifact.

When we evaluate the MLP strictly within attack sessions, it achieves a pooled balanced accuracy of 0.942 (macro-averaged 0.906). However, a trivial baseline rule—"classify window as attack if it contains at least one IP PacketIn"—achieves a pooled balanced accuracy of 0.995 and a macro-averaged score of 1.000 across the exact same windows (Fig. 3). This 0.995 score reflects the labeling protocol combined with session separation: because the ambiguity mask excludes attack-session windows containing unlabelled IP traffic, the surviving non-attack windows consist almost entirely of non-IP management traffic (317 of 320 windows are ARP or LLDP). Thus, any rule checking for the presence of IP packets achieves near-perfect separation inside attack sessions without learning any attack characteristics.

B. Session Probes

To test whether classifiers recognize session identity rather than attack signatures, we trained an MLP on windows where all attack-matched PacketIns were excised (Table VII). The model predicts whether a window originates from an attack session with 0.940 balanced accuracy (D1). When restricted to attack-free windows (D2), it achieves 0.950. However, 82% of these residual windows contain unmatched IP traffic that likely represents unlabelled attack remnants, and D2 rests on only 6 windows from 2 captures. These findings demonstrate that in non-interleaved benchmark recordings, attack detection cannot be cleanly disentangled from session fingerprinting.

VIII. EVALUATION-VALIDITY AUDIT: RANDOM FLOW SPLITS VS. HELD-OUT HOSTS

A. Experimental Design Across Two Benchmarks

To rigorously evaluate data leakage in flow-based evaluations, we implemented a standardized evaluation pipeline applied to two prominent benchmarks: InSDN (343 889 flows) and an external reference dataset, CICIDS2017 (2 100 814 flows). We evaluate an identical GPU-accelerated XGBoost classifier

TABLE V
WITHHELD ATTACK FAMILY (BACC, W=1 S, STRICT LABELS). "BEST THR.": BEST OF FOUR THRESHOLDS, CHOSEN PER FAMILY.

Withheld family	MLP	Logistic	Best thr.
Flood (DoS+DDoS)	0.936±0.011	0.953	0.814
Probe	0.997±0.000	0.988	0.970
Web	0.904±0.020	0.909	0.699
Brute force	0.891±0.029	0.913	0.888
Exploit	0.929±0.037	0.997	0.929
Botnet	0.932±0.024	0.803	0.965

TABLE VI
CROSS-DEPLOYMENT TRANSFER (BACC, STRICT LABELS).

Train → test	MLP	Logistic	Best thr.
Metasploitable-2 → OVS	0.905±0.021	0.928	0.866
OVS → Metasploitable-2	0.990±0.001	0.985	0.947

TABLE VII
SESSION PROBE: PREDICTING BENIGN VS. ATTACK *session* FROM PACKETIN FEATURES AFTER REMOVING ATTACK-MATCHED PACKETINS (≥5 REMAINING PER WINDOW). THE RESIDUAL CAN STILL CONTAIN UNLISTED ATTACK TRAFFIC (D1: 424 OF 518 ATTACK-SESSION WINDOWS CONTAIN UNMATCHED-IP PACKETINS), AND D2 RESTS ON 6 WINDOWS FROM 2 CAPTURES.

Variant	BAcc	Majority
All windows (D1)	0.940±0.005	0.500
Attack-free windows only (D2)	0.950±0.031	0.500

TABLE VIII
EVALUATION-VALIDITY AUDIT: BALANCED ACCURACY (BENIGN VS. ATTACK) OF THE SAME XGBOOST (GPU) ON THE 77 NUMERIC CICFLOWMETER FEATURES, SAME PIPELINE ON BOTH DATASETS, 10 SEEDS (MEAN±95% CI OVER SEEDS). A, B AND C USE NO IPS, PORTS OR TIMESTAMPS; D USES ONLY THOSE. ROWS: INSDN 343 889, CICIDS2017 2 100 814. C IS A SINGLE DETERMINISTIC SPLIT.

Protocol	InSDN	CICIDS2017
A. Random 70:30 flow split	0.9998	0.9917±0.0001
B. Held-out source hosts (70:30 by source IP)	0.907±0.085	0.641±0.056
C. Per-file time-ordered 70:30 (deterministic)	0.9997	0.7187
D. IPs, ports and Flow ID only, random split	1.0000	0.9999
E. Split-A test rows duplicating a train row (%)	48.7	20.3

across four distinct protocols using 10 random seeds (Table VIII and Fig. 4):

- **Protocol A (Random 70:30 Flow Split):** Standard randomized flow-level partition. Excludes all IP addresses, ports, Flow IDs, and timestamps (retaining 77 numeric features).
- **Protocol B (Held-Out Source Hosts):** 70:30 partition grouped by source IP. No source host appears in both training and test sets.
- **Protocol C (Time-Ordered Split):** Deterministic 70:30 chronological partition per capture file.
- **Protocol D (Identifier-Only Split):** Random 70:30 split trained exclusively on IP addresses, port numbers, and Flow IDs.
- **Protocol E (Redundancy Audit):** Measurement of the percentage of test-set rows that possess an exact feature duplicate in the training set under Protocol A.

B. The Generalization Collapse on Unseen Hosts

Under Protocol A (random flow split), XGBoost achieves near-perfect balanced accuracy: 0.9998 on InSDN and 0.9917 ± 0.0001 on CICIDS2017. However, under Protocol B (held-out source hosts), performance on InSDN drops by nine percentage points to 0.907 ± 0.085, with individual splits swinging from 1.00 down to 0.66. Across the 10 random seeds on InSDN, the balanced accuracy distribution is widely dispersed: five seeds score near 1.00 (0.996, 0.998, 0.999, 1.000, 1.000), two score between 0.85 and 0.94 (0.859, 0.939), and three drop below 0.84 (0.660, 0.786, 0.837), consistent with the acute host concentration documented below. On CICIDS2017, the drop is catastrophic: performance collapses by 35 percentage points to 0.641 ± 0.056. A per-file time-ordered split shows no drop on InSDN (0.9997), where attacks were generated continuously within sessions, but falls to 0.719 on CICIDS2017, where attacks were executed in episodic time windows.

Under Protocol D, a classifier supplied exclusively with network identifiers (IP addresses, ports, and Flow ID) achieves 1.0000 on InSDN and 0.9999 on CICIDS2017. This demonstrates that network identifiers alone provide complete class separability without requiring any statistical traffic features.

C. The Role of Data Redundancy and Class Coverage

Under Protocol E, we discover that 48.7% of random-split test rows in InSDN (and 20.3% in CICIDS2017) are exact duplicates of training rows. To test whether duplicate flows drive the random-split score, we deduplicated InSDN completely, reducing the dataset from 343 889 to 184,607 rows. On this deduplicated dataset, the random-split score remains unchanged at 0.9998. Thus, feature redundancy alone does not explain the performance gap.

To ensure that host grouping does not artificially penalize performance by withholding entire attack classes from training, we enforced a class-coverage constraint: holding out source hosts while guaranteeing that every attack class appears in the training partition. Under this controlled setting, balanced accuracy is 0.939 (0.812 after deduplication). Grouping by the 3-tuple `(src_ip, dst_ip, dst_port)` yields 0.961 (0.984 after deduplication). The triple-grouped split fragments traffic while retaining communication from the same attacker hosts on different ports across partitions, allowing the model to leverage host-level similarities without exact flow duplication.

D. Root-Cause Analysis: Extreme Host Concentration

Why do held-out-host scores exhibit such dramatic variance? Analyzing InSDN's topological composition provides an explanation consistent with this dispersion: InSDN contains 122,868 distinct source IP addresses; however, 122,307 of these source IPs appear in exactly one flow record, generated during spoofed DoS/DDoS flooding. Crucially, only eight hosts account for 500 or more flows, yet these eight hosts encompass 215,404 of InSDN's 343,889 total rows. Among these eight dominant

TABLE IX
FEATURE-RECIPE REPLICATION ON INSDN (ONE XGBOOST CLASSIFIER, GPU, 80:20, 10 SEEDS, MEAN±95% CI): BINARY BALANCED ACCURACY AND 8-CLASS ACCURACY UNDER A RANDOM FLOW SPLIT AND A SOURCE-IP-GROUPED SPLIT. RECIPES APPROXIMATE FEATURE CHOICES DESCRIBED IN PUBLISHED INSDN PAPERS; THEY ARE NOT THOSE PAPERS' MODELS OR CODE. GROUPING BY SOURCE HOST ALSO CHANGES THE ATTACK-CLASS MIX BETWEEN TRAIN AND TEST, SO GROUPED VALUES ARE A STRESS TEST.

Recipe	BAcc random	BAcc grouped	8-cls random	8-cls grouped
Numeric features only	0.9998±0.0000	0.943±0.057	0.9991±0.0001	0.788±0.186
Numeric + ports	0.9998±0.0000	0.856±0.151	0.9998±0.0000	0.687±0.241
Numeric + identifier columns	1.0000±0.0000	0.941±0.088	0.9999±0.0000	0.804±0.187
Top-15 by MI, selected before split	1.0000±0.0000	0.941±0.088	0.9999±0.0000	0.835±0.175

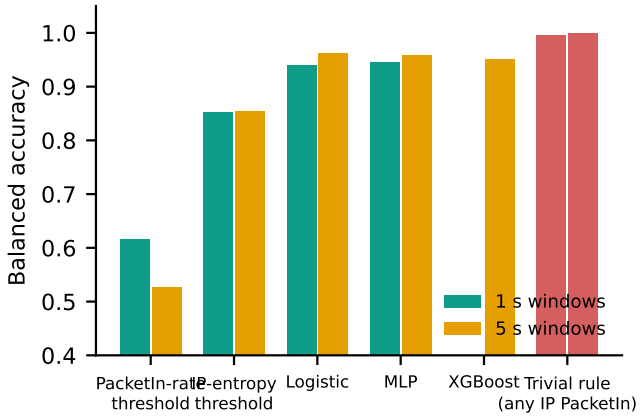


Fig. 3. Balanced accuracy on unseen captures under capture-grouped CV across 10 seeds (XGBoost evaluated at 5 s). The red bars represent the trivial rule ("at least one IP PacketIn"), which matches or outperforms all machine learning models within attack sessions.

hosts, exactly three are attacker hosts. Consequently, a source-IP partition simply determines which side of the split these few dominant hosts fall onto, creating widely dispersed outcomes where scores swing between 1.00 and 0.66, consistent with the partition of dominant hosts.

To quantify generalization to individual hosts, we conducted a Leave-One-Host-Out (LOHO) evaluation, holding out one dominant host at a time while training on all remaining records. When holding out the primary attacker (host 200.175.2.130), the model catches only 51.6% of its attack flows (DoS detection recall collapses to 34.7%). Conversely, when holding out the largest benign host (host 192.168.20.133), the false positive rate spikes to 37.9%. On CICIDS2017, which contains 48 source IPs (15 with ≥ 500 flows), holding out the primary attacker (172.16.0.1, 442 098 flows) results in an attack recall of only 28.3% and a benign false positive rate of 54.0%. Individual bot hosts held out in isolation achieve detection rates between 23.4% and 43.5%.

E. Replication of Published Pipeline (SDNGuardStack)

We partially replicated the published SDNGuardStack pipeline [5] using our own implementation across 5 seeds. Under a random split, the model reproduces its published performance, achieving 0.9999 accuracy (published: 99.98%). However, under held-out source hosts, mean accuracy drops

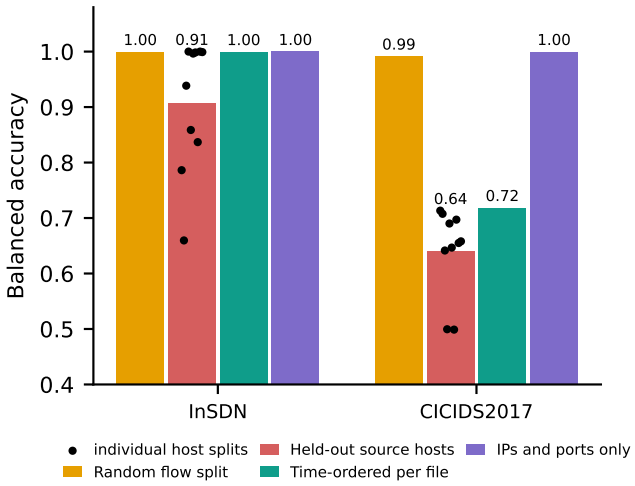


Fig. 4. Balanced accuracy across four evaluation protocols on InSDN and CICIDS2017 (10 seeds; dots represent individual host splits). When source hosts are held out, performance degrades sharply, and identifier-only models achieve near-perfect scores.

to 0.72 when selecting features strictly on training folds (per-seed: 0.24, 0.73, 1.00, 0.66, 0.99) and 0.78 when selecting on all rows (per-seed: 0.33, 0.99, 1.00, 0.62, 0.98); 95% confidence intervals span up to ±0.39. This dispersion is directly attributable to InSDN's host concentration: because a 20% host partition draws hosts with vastly unequal traffic volumes, the test row share varies from 8% to 40% across seeds, and the lowest score (0.24) occurs in the run with the largest test partition (40% of all rows), consistent with the test fold being dominated by a few large hosts while no test class is absent from training (`unseen_test_share` is 0.0 across all seeds).

IX. A FORWARDING-APPLICATION CASE STUDY

InSDN shows one forwarding configuration. To see whether what the controller observes depends on the forwarding application, we built a small testbed (Mininet, one Open vSwitch switch, six hosts; a Ryu OpenFlow 1.3 learning switch with a configurable entry granularity) and recorded its control channel with the same parser. The three modes install, for each learned destination, an entry matching (i) the MAC pair; (ii) the 5-tuple for TCP/UDP with a 10 s idle timeout; or (iii) the 5-tuple with a 2 s idle timeout applied to all entries. In the two 5-tuple modes, other IP traffic receives an (src, dst, protocol) entry and ARP an ARP-only entry, so a coarse entry for one protocol cannot match another. The controller polls flow statistics every second. Traffic is benign only and generated by the same program with the same random seeds in every mode (request timing still differs): five hosts repeatedly fetch files of 16 KB to 1 MB from an HTTP server (80%) or ping it (20%) with randomized uniform pacing (0.05–0.8 s) for 90 s (captures span ∼99 s including start-up). Each mode was run 10 times (seeds 0–9), reporting mean and 95% t-interval over runs (Table XI); the 30 runs completed 26 891 requests, of which 41 failed (below). We repeated the study on a loop-free three-switch chain (Table XII, below).

TABLE X

TARGETED LITERATURE AUDIT OF REPRESENTATIVE SDN INTRUSION DETECTION EVALUATIONS ON BENCHMARK DATASETS (12 PAPERS EXAMINED, 10 READ IN FULL TEXT; DETAILS IN `LITERATURE_CHECK.JSON`). NONE OF THE INSDN PAPERS READ IN FULL REPORTS A HOST-GROUPED OR CAPTURE-GROUPED SPLIT, AND THREE USE IDENTIFIER ATTRIBUTES AS INPUT FEATURES.

Paper	Venue	Dataset	Evaluated Model	Split Protocol	Identifiers Used?	Methodological Findings / Red Flags
Towhid & Shahriar [28]	Preprint 2023	InSDN	RF / LSTM	Random 70:30	Excluded	Divides all flows 70:30; no host/session grouping; offline Scapy features
Ataa et al. [4]	Sci. Rep. 2024	InSDN	CNN-LSTM / Transf.	Shuffled (ratio unstated)	Excluded	Shuffled flows; no train/test ratio or grouping stated; single seed
Abdallah et al. [29]	ACM ARES 2021	InSDN	Hybrid CNN-LSTM	Unverified (paywall)	Unverified	Publisher access restricted; split details unverified
Elsayed et al. [30]	ASEJ 2023	InSDN / ToN-IoT	SATIDS (LSTM)	Unverified (paywall)	Unverified	Publisher access restricted; split details unverified
OptiXGB-IDS [8]	Sci. Rep. 2026	InSDN / CIC-17	OptiXGB + WGAN-GP	Random 80:20	Src/Dst Port kept	Preprocessing leakage-free, but flow split random; Dst Port major tree split
Alzahrani et al. [9]	PeerJ CS 2025	InSDN / Kaggle	RF + Deep Extractor	train_test_split (random)	Unstated	scikit-learn random split; no grouping; row count mismatch unexplained
Sajid et al. [10]	PLOS ONE 2025	InSDN / CICDDoS	CNN-MLP	Unstated validation	Dst Port kept	Dst Port top SHAP feature; no split ratio or host grouping reported
Al-Haija & Smadi [31]	PLOS ONE 2025	UNSW / BoT-IoT	GRU-NTM	Chronological 80:20	Unstated	Chronological split, but validation set used for early stopping/tuning
Ashikuzzaman et al. [5]	arXiv 2026	InSDN	Stacking + LightGBM	Random 80:20	Flow ID, IPs, Ports	Top mutual-information features include Flow ID, Src IP, Dst IP, Src Port
Malik et al. [32]	Submission 2026	InSDN	BiLSTM + RFE	Stratified 5-fold CV	Unstated	Stratified 5-fold drops score to 81.60%; attributed to split variance
Kumar et al. [33]	Sci. Rep. 2026	InSDN	Quantum + MSDC Net	Random 80:20	Unstated	Evaluated 4 split ratios; picked 80:20 based on test set; no grouping
Saleh & Hamad [34]	JESA 2026	SICA / InSDN	DNN / LSTM	Random 70:20:10	Excluded	Removed Flow ID/IPs, yet InSDN scores 99.61% under random flow split

TABLE XI

FORWARDING APPLICATION VS. WHAT THE CONTROLLER OBSERVES: BENIGN TRAFFIC FROM THE SAME GENERATOR AND SEEDS (5 HOSTS → 1 HTTP SERVER, PLUS PINGS), 1-SWITCH MININET/RYU TESTBED, ∼99 S CAPTURE PER RUN, 10 RUNS PER MODE, MEAN±95% CI. ENTRIES = FLOW ENTRIES RETURNED PER FLOW-STATISTICS REPLY (1 S POLLING).

Mode	TCP PacketIns	All PacketIns	FlowMods	MAC-pair entries	IP-specific entries
MAC-pair (as ONOS in InSDN)	12.3±2.3	76.8±1.8	19.1±1.8	9.5±0.0	0.0±0.0
5-tuple, 10 s idle	10422.0±202.0	10497.0±204.7	10442.8±200.8	0.0±0.0	153.1±2.4
5-tuple, 2 s idle	10391.6±249.4	10611.3±258.7	10556.4±252.3	0.0±0.0	36.9±0.5

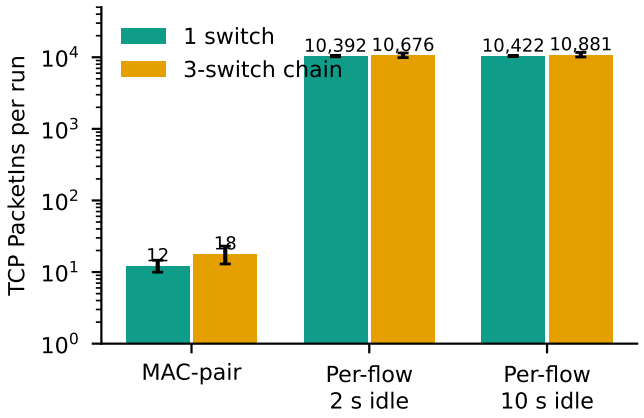


Fig. 5. TCP PacketIns per run (mean ± 95% CI, 10 runs per mode, log scale) under three forwarding modes, on one switch and on a three-switch chain. Data: `live_modes_v4_one10.json`, `live_modes_v4_linear3.json`.

Granularity sets the PacketIn volume. MAC-pair mode produced 12.3±2.3 TCP PacketIns per run, against 10 422±202 (10 s idle) and 10 392±249 (2 s idle) under per-flow forwarding: 665× to 1 752× (10 s idle) and 668× to 1 780× (2 s idle) more in each of the ten paired runs (larger in 10 of 10; two-sided sign-test $p = 0.002$). That is about 11–12 TCP PacketIns per completed request, so each connection is punted repeatedly; the most likely cause is packets that arrive before the FlowMod takes effect (we install entries without a barrier and send full packets), but we did not isolate this mechanism. FlowMods (19.1 vs. about 10 440 and 10 560 per run) track the PacketIns and are not independent evidence. The MAC-pair result is a replication: the same mode in the first, flawed campaign gave 11.0 ± 4.1. Seeds 0–4 are the earlier five runs per mode (reported in the previous version of this study); seeds 5–9 were added later.

Idle timeout sets what a poller sees. Per-flow entries returned per statistics reply fell from 153.1 ± 2.4 to 36.9 ± 0.5 when the idle timeout dropped from 10 s to 2 s, a ratio of 0.234 to 0.246 in all ten pairs: entries expire between polls, so a 1 s poller sees about a quarter as many. The result is close to what a shorter timeout implies; its value is the size of the loss for a realistic polling interval. In MAC-pair mode the replies contain only the table-miss rule and MAC-pair entries (9.5 ± 0.03 per reply); no entry matches IP or transport fields, as in InSDN (whose ONOS additionally installed `eth_type` punt rules).

A flawed first campaign. Our first application installed MAC-pair entries for ping and ARP in every mode; these also match later TCP between the same hosts, so most connections never reached the controller and per-flow mode produced only 43.4 ± 30.1 TCP PacketIns per run (4× MAC-pair) with a wide interval. The original application is archived as `ryu_fwd_v1.py`. We report it because it shows how an overlapping coarse entry hides traffic from the controller by two orders of magnitude, independently of the forwarding mode's nominal granularity.

Reliability. One run (5-tuple, 2 s idle, seed 4) had 41 failed requests (7–10 per client); the other 29 one-switch runs and all 30 three-switch runs had none. The run did not reproduce, and its failures were spread evenly over all five clients, which is consistent with a short stall of the shared path or controller rather than one client. The cause is not established (the failure reasons were not logged in that run; later runs log them). The run is included in all figures.

A second topology. On a three-switch chain (hosts h1–h6, server two hops from h1 and h2; 10 runs per mode, 26 826 requests, none failed) the contrast holds (Table XII): 18.0 ± 5.0 TCP PacketIns per run under MAC-pair against $10\,881 \pm 794$ and $10\,676 \pm 804$ under per-flow forwarding (391× to 1 509× in every paired run), and the per-flow entries a 1 s poller sees per reply fall from 112.4 ± 2.3 to 27.0 ± 0.4 (ratio 0.235 to

0.245). The ratio is smaller than on one switch because the MAC-pair baseline rises with the extra hops (18.0 vs. 12.3).

A stock Ryu application. To check that the MAC-pair behaviour is not an artefact of our own application, we ran Ryu's unmodified `simple_switch_13` on the same one-switch topology (5 runs of 60 s; 2 982 requests, none failed). Of 91 FlowMods, 86 match `(in_port, eth_dst, eth_src)` and 5 are the table-miss rule; none matches IP or transport fields. The controller received 71, 69, 76, 76 and 76 PacketIns per run (mean 73.6, all protocols). Like ONOS's `fwd`, this application installs MAC-pair entries; this follows from the application's code, and it does not test OpenDaylight, ONOS under other applications, or any production deployment.

Scope. This is a testbed with at most three switches with one forwarding application and benign traffic only; we run no attacks or detectors in it. The mode contrast does not transfer to InSDN's ONOS, which punted about one packet per target packet even with MAC-pair entries (§IV), and no claim is made about detection accuracy under any forwarding mode.

X. Targeted Literature Audit and Feature-Recipe Evaluations

A. Targeted Audit of SDN-IDS Literature

To assess how prevalent these evaluation flaws are in the SDN-IDS literature, we conducted a targeted audit of 12 representative papers (eleven focusing on SDN or InSDN, and one non-SDN control). As documented in Table X, ten papers were read in full text.

Our audit reveals widespread methodological vulnerabilities:

1) **Absence of Host Grouping:** Among all InSDN papers read in full, none reports a host-grouped or capture-grouped validation split. Most employ simple randomized partitions (such as `train_test_split`) or fail to document their split mechanism entirely.
2) **Use of Network Identifiers as Model Inputs:** Three papers explicitly include non-generalizable network identifiers (Flow IDs, IP addresses, or port numbers) among their feature sets. Notably, SDNGuardStack [5] ranks Flow ID, Source IP, and Destination IP as the most important features via mutual information and SHAP analysis. Similarly, OptiXGB-IDS [8] and Sajid et al. [10] retain destination port, which emerges as the dominant decision feature.
3) **Offline Evaluation of Controller Claims:** Multiple papers frame their models as lightweight controller-side defenses, yet evaluate exclusively on offline CSVs without deploying within an SDN controller.

B. Feature-Recipe Evaluations on InSDN

To determine whether the evaluation gap is specific to feature selection, we implemented four generic feature recipes approximating choices in the literature, evaluated with GPU XGBoost across 10 seeds (Table IX):

1) **Numeric features only (77 features):** Random split achieves 0.9998 BAcc (0.9991 8-class); held-out source hosts drops to 0.943 BAcc (0.788 8-class).
2) **Numeric + ports (79 features):** Random split achieves 0.9998 BAcc (0.9998 8-class); held-out source hosts drops to 0.856 BAcc (0.687 8-class).
3) **Numeric + identifiers (83 features):** Random split achieves 1.0000 BAcc (0.9999 8-class); held-out source hosts drops to 0.941 BAcc (0.804 8-class).
4) **Top-15 by Mutual Information:** Selected on full data prior to splitting. Mutual information automatically selects Flow ID, Src IP and Dst IP. Random split achieves 1.0000 BAcc (0.9999 8-class); held-out hosts drops to 0.941 BAcc (0.835 8-class, widest CI 0.241).

Across all recipes, random splits achieve near-perfect scores ($\geq$ 0.9998), but performance degrades under host-grouped evaluation. Under host grouping, adding identifier columns yields 0.941, comparable to numeric features alone (0.943), which is consistent with decision trees relying on numeric features when test-set IPs are unseen. However, under random splits, computing mutual information across the unpartitioned dataset selects Flow ID, Src IP and Dst IP among the top features, enabling near-perfect scores (1.0000) by memorizing training-set identifiers across partitions.

XI. Methodological Guidelines and Evaluation Checklist

To improve rigor in SDN intrusion detection research, we propose six concrete methodological guidelines:

1) **Verify Controller Observability:** Researchers proposing controller-side defenses must verify that input features correspond to OpenFlow control-plane primitives. If features require full-packet timing moments, the architectural cost of data-plane polling or flow-table modifications must be explicitly accounted for.
2) **Enforce Topology-Aware and Host-Disjoint Splits:** Standard random flow splits should not be used to evaluate generalizable detection. Datasets must be partitioned such that source hosts in the test set are completely withheld from training ($\mathcal{H}_{\text{train}} \cap \mathcal{H}_{\text{test}} = \emptyset$).
3) **Purge Non-Generalizable Network Identifiers:** All non-statistical network metadata (such as Flow IDs, IP addresses, MAC addresses, and ephemeral transport ports) must be removed prior to feature extraction.
4) **Audit Dataset Host Concentration:** Authors must report the distribution of flows per host. In datasets where a handful of hosts dominate total volume, Leave-One-Host-Out (LOHO) testing should be reported alongside aggregate metrics.
5) **Perform Deduplication and Redundancy Audits:** Cross-split duplicate flows must be identified and removed prior to model training to prevent trivial memorization.
6) **Evaluate Forwarding Application Sensitivity:** Detectors relying on control-plane metrics (such as PacketIn rates) must evaluate sensitivity to forwarding application policies, entry granularity, and idle timeout configurations.

TABLE XII
FORWARDING APPLICATION VS. WHAT THE CONTROLLER OBSERVES: BENIGN TRAFFIC FROM THE SAME GENERATOR AND SEEDS (5 HOSTS → 1 HTTP SERVER, PLUS PINGS), 1-SWITCH MININET/RYU TESTBED, ∼99 S CAPTURE PER RUN, 10 RUNS PER MODE, MEAN±95% CI. ENTRIES = FLOW ENTRIES RETURNED PER FLOW-STATISTICS REPLY (1 S POLLING).

Mode	TCP PacketIns	All PacketIns	FlowMods	MAC-pair entries	IP-specific entries
MAC-pair (as ONOS in InSDN)	18.0±5.0	267.8±18.2	33.9±3.8	7.0±0.0	0.0±0.0
5-tuple, 10 s idle	10881.3±793.8	11159.9±795.4	10928.3±791.8	0.0±0.0	112.4±2.3
5-tuple, 2 s idle	10675.5±803.9	11275.8±814.7	11040.4±805.9	0.0±0.0	27.0±0.4

XII. DISCUSSION AND LIMITATIONS

A. Scope and Experimental Boundaries

Our evaluation-validity audit utilized an optimized XGBoost classifier to isolate protocol effects across identical feature spaces; other neural architectures may exhibit varying degrees of sensitivity. While we evaluated two major benchmarks (InSDN and CICIDS2017), examining additional datasets is desirable. We attempted to evaluate a third candidate, UKMDDoSDN [35], but found that its public PCAPs contained no benign traffic (Appendix A). We frame this work as an empirical measurement and methodological critique of benchmark evaluation practices, rather than a claim of a deployable end-to-end intrusion detection system.

B. Ground-Truth Labeling Trade-Offs

Strict 5-tuple matching ensures that controller windows are reliably grounded in benchmark labels, but discards unlabelled traffic in attack sessions. Under an alternative broader rule that labels any window containing IP addresses observed during attacks, overall balanced accuracy is lower (MLP 0.881 at 1 s, 0.887 at 5 s). However, model rankings remain consistent, and transfer from OVS to Metasploitable-2 remains strong (0.977 vs. 0.924).

C. Deployment Feasibility

An operational controller-side detector would inspect control-channel traffic via port mirroring or controller event handlers. However, as demonstrated in Section IX, forwarding application changes dynamically alter PacketIn rates by two to three orders of magnitude. Consequently, control-plane detectors must be dynamically recalibrated whenever forwarding policies change.

XIII. CONCLUSION

This study demonstrated that benchmark evaluations in SDN intrusion detection suffer from critical observability and evaluation-validity disconnects. In InSDN, the ONOS controller never installed or reported 5-tuple flow entries; flow table entries matched exclusively on MAC pairs or EtherTypes. While models operating on controller-visible PacketIn streams achieve 0.946 balanced accuracy on unseen captures, this performance is confounded by non-interleaved benchmark session design, where a trivial IP PacketIn rule achieves 0.995.

Furthermore, we demonstrated that reported 99.9%+ benchmark accuracies are inflated relative to host-held-out evaluation on highly concentrated host topologies. On InSDN and CICIDS2017, holding out source hosts drops performance to 0.907 and 0.641, respectively. In leave-one-host-out stress tests, detection recall on the primary attacker collapses to 51.6% (DoS recall 34.7%), while flagging 37.9% of benign host flows. Our targeted audit of 12 representative papers confirmed that none employed host-grouped splits, and three utilized network identifiers as model inputs. By releasing our parser, replication pipelines, and evaluation framework, we provide the community with tools to conduct physically grounded, leakage-controlled SDN intrusion detection research.

APPENDIX A
CANDIDATE SECOND DATASETS EXAMINED

In addition to InSDN and CICIDS2017, we examined two candidate datasets for control-channel analysis. For UKMDDoSDN [35], we fetched `syn_flood_combined.pcap` (18 922 240 B; 199 273 packets) and `udp_flood_combined.pcap` (15 420 748 B; 110 518 packets). Parsing confirmed that neither capture contains packets on TCP port 6633 or 6653; they represent data-plane host captures without OpenFlow control traffic, and contain no benign class. For LAN-SDN-NIDS [36], its record describes flow-level records with 11 control-plane metrics, but raw OpenFlow control-plane captures are not available. Neither dataset could support control-channel reassembly.

DATA AND CODE AVAILABILITY

InSDN CSVs and PCAPs are publicly available (sources and SHA-256 digests in `data/MANIFEST.md`). Our OpenFlow 1.3 parser, feature extraction pipeline, and evaluation scripts are available in the project repository. The reproduction script `research/insdn_controller/reproduce.sh` runs all experiments and verifies numerical parity against stored result files. The test script `paper/verify_numbers.py` verifies all numerical claims in the text. Recorded capture hashes are documented in `capture_hashes.sha256`.

ACKNOWLEDGMENT

All three authors contributed equally to the study design, empirical analysis, and manuscript preparation. An AI assistant (Claude, Anthropic) was utilized to assist with scripting, literature synthesis, and drafting; the authors independently verified all experimental results against stored result files and assume full responsibility for the manuscript.

REFERENCES

[1] D. Kreutz, F. M. V. Ramos, P. E. Veríssimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-defined networking: A comprehensive survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2015.

[2] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008.

[3] M. S. Elsayed, N.-A. Le-Khac, and A. D. Jurcut, "InSDN: A novel SDN intrusion dataset," *IEEE Access*, vol. 8, pp. 165 263–165 284, 2020.

[4] M. Ataa, E. Sanad, and R. El-khoribi, "Intrusion detection in software defined network using deep learning approaches," *Scientific Reports*, vol. 14, no. 1, p. 26842, 2024.

[5] M. Ashikuzzaman, M. M. Rahman, and M. S. Islam, "SDNGuardStack: An explainable ensemble learning framework for high-accuracy intrusion detection in software-defined networks," *arXiv preprint arXiv:2604.20934*, 2026.

[6] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.

[7] A. H. Lashkari, G. Draper-Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Tor traffic using time based features," in *Proc. 3rd Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2017, pp. 253–262.

[8] Z. Ahmad, A. Alghamdi, I. Ullah, and M. Raza, "An efficient and interpretable intrusion detection framework for software-defined networks with multi-class imbalance," *Scientific Reports*, vol. 16, p. 58514, 2026.

[9] A. Alzahrani, F. Al-Alami, and S. Al-Subaie, "Software defined network intrusion system to detect malicious attacks in computer Internet of Things security," *PeerJ Computer Science*, vol. 11, p. e3103, 2025.

[10] M. Sajid, U. Tariq, and A. K. Bashir, "Distributed denial of services (DDoS) attack detection in SDN using optimizer-equipped CNN-MLP," *PLOS ONE*, vol. 20, no. 2, p. e0312425, 2025.

[11] Open Networking Foundation, "OpenFlow switch specification, version 1.3.0," ONF TS-006, 2012.

[12] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2010, pp. 305–316.

[13] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symposium*, 2022, pp. 3971–3988.

[14] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: the CICIDS2017 case study," in *Proc. IEEE Security and Privacy Workshops (SPW)*, 2021, pp. 7–12.

[15] R. Flood, G. Engelen, D. Aspinall, and L. Desmet, "Bad design smells in benchmark NIDS datasets," in *Proc. 9th IEEE European Symposium on Security and Privacy (EuroS&P)*, 2024, pp. 420–436.

[16] S. Roy, A. Paul, and D. Das, "Unmasking shortcut learning in IoT intrusion detection," 2026, arXiv:2609.28725.

[17] M. Sarhan, S. Layeghy, and M. Portmann, "Towards a standard feature set for network intrusion detection system datasets," *Mobile Networks and Applications*, vol. 27, no. 1, pp. 357–370, 2022.

[18] D. Pinto, J. Vitorino, E. Maia, I. Amorim, and I. Praça, "Flow exporter impact on intelligent intrusion detection systems," in *Proc. 11th Int. Conf. Information Systems Security and Privacy (ICISSP)*, vol. 2, 2025, pp. 289–298.

[19] A. Pekar and R. Jozsa, "Early-stage anomaly detection: A study of model performance on complete vs. partial flows," *IEEE Transactions on Network and Service Management*, vol. 22, pp. 1–12, 2025, arXiv:2407.02856.

[20] ——, "Evaluating ML-based anomaly detection across datasets of varied integrity: A case study," 2024, arXiv:2401.16843.

[21] M. Lanvin, P.-F. Gimenez, Y. Han, F. Majorczyk, L. Mé, and E. Totel, "Errors in the CICIDS2017 dataset and the significant differences in detection performances it makes," in *Proc. Int. Conf. Risks and Security of Internet and Systems (CRiSIS)*, 2022, pp. 18–33.

[22] L. Liu, G. Engelen, T. Lynar, D. Essam, and W. Joosen, "Error prevalence in NIDS datasets: A case study on CIC-IDS-2017 and CSE-CIC-IDS-2018," in *Proc. IEEE Conf. Communications and Network Security (CNS)*, 2022, pp. 280–288.

[23] R. Braga, E. Mota, and A. Passito, "Lightweight DDoS flooding attack detection using NOX/OpenFlow," in *Proc. IEEE Local Computer Network Conference (LCN)*, 2010, pp. 408–415.

[24] S. M. Mousavi and M. St-Hilaire, "Early detection of DDoS attacks against SDN controllers," in *Proc. Int. Conf. Computing, Networking and Communications (ICNC)*, 2015, pp. 77–81.

[25] S. Shin, V. Yegneswaran, P. Porras, and G. Gu, "AVANT-GUARD: Scalable and vigilant switch flow management in software-defined networks," in *Proc. ACM Conf. Computer and Communications Security (CCS)*, 2013, pp. 413–424.

[26] H. Wang, L. Xu, and G. Gu, "FloodGuard: A DoS attack prevention extension in software-defined networks," in *Proc. 45th Annual IEEE/IFIP Int. Conf. Dependable Systems and Networks (DSN)*, 2015, pp. 239–250.

[27] M. Dhawan, R. Poddar, K. Mahajan, and V. Mann, "SPHINX: Detecting security attacks in software-defined networks," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2015.

[28] M. S. Towhid and N. Shahriar, "Early detection of intrusion in SDN," University of Regina Preprint, 2023, https://uregina.ca/~nss373/papers/Early-detection-SDN.pdf.

[29] N. Abdallah, N.-A. Le-Khac, A. Jahromi, and A. D. Jurcut, "A hybrid CNN-LSTM based approach for anomaly detection systems in SDNs," in *Proc. 16th Int. Conf. Availability, Reliability and Security (ARES)*, 2021, pp. 1–8.

[30] M. S. Elsayed, M. Hamada, M. Abdalla, and A. Elsaid, "Securing IoT and SDN systems using deep-learning based automatic intrusion detection," *Ain Shams Engineering Journal*, vol. 14, no. 10, p. 102211, 2023.

[31] Q. A. Al-Haija and M. Smadi, "Deep memory for deep threats: A novel architecture combining GRUs and deep learning models for IDS," *PLOS ONE*, vol. 20, no. 3, p. e0332752, 2025.

[32] M. S. Malik and M. Ahmed, "Improving robustness and interpretability of deep learning-based intrusion detection in software-defined networks," *Preprint*, 2026.

[33] R. Kumar, J. Singh, and S. Sharma, "A federated deep learning approach for SDN security with quantum optimized feature selection and hybrid MSDC net architecture," *Scientific Reports*, vol. 16, p. 12957, 2026.

[34] A. Saleh and T. Hamad, "SICA: Generation and deep learning-based evaluation of a novel dataset for intrusion detection in SDN-IoT environments," *Journal of European Systems Automated (JESA)*, vol. 59, no. 1, pp. 115–128, 2026.

[35] M. A. B. Mokti, "UKMDDoSDN: A DDoS attacks dataset for software-defined networks, version 2," Mendeley Data, 2025.

[36] J. Arévalo-Herrera, J. E. Camargo, J. I. Martínez Torre, J. M. Ramírez, and T. Zona-Ortiz, "A multi-class SDN intrusion detection dataset with synchronized OpenFlow control-plane telemetry," *Data*, vol. 11, no. 8, p. 195, 2026.